

Huffman Codes

Nick Arnosti

April 28, 2019

1 Motivation

Suppose that you want to convert a message consisting only of the letters a, b, c, d into a sequence of bits (zeros and ones). One simple way to do it is to represent a with 00, b with 01, c with 10 and d with 11. If you do this, your message will always have exactly twice as many bits as letters.

It may be possible to make your message shorter. For example, suppose that a occurs more frequently than the other letters: half of all letters are a , one quarter are b , and the remaining one quarter are evenly divided between c and d . Then we could use the following *variable length* code: a is 0, b is 10, c is 110, and d is 111. Under this code, we will average only $1 \times \frac{1}{2} + 2 \times \frac{1}{4} + 3 \times \frac{1}{8} + 3 \times \frac{1}{8} = 1.75$ bits per letter, resulting in a 12.5% reduction in message size.

Of course, if we use the second code but the letters are actually equally frequent, then we will spend an average of 2.25 bits per letter, and would have been better off sticking with the original scheme. Therefore, the most efficient code depends on the frequencies of the letters. This post discusses two questions. First, how can we find the most efficient code for a given message? Second, in this most efficient code, how will the frequency of letters relate to the number of bits used to represent them?

2 Decodability and Prefix-Free Codes

Say that we have an alphabet $\mathcal{A}$. We wish to assign each element of $\mathcal{A}$ to a sequence of bits – let $\mathcal{B}$ be the set of finite sequences of bits. A *code* is a mapping from $\mathcal{A}$ to $\mathcal{B}$. We call a sequence of bits associated with a letter a *codeword*.

Any good encoding can be decoded. Say that a code is *decodable* if given any two distinct sequences of letters lead to different sequences of bits. Note that this means more than just that each letter gets a different codeword, as we are not allowed “spaces” to separate letters. For example, if we choose to represent a with 0, b with 1, and c with 01, then every time we see 01, we won’t know whether it represents c or ab .

We might also wish for our code to be “easy” to decode. One way of formalizing this is to require that the code be *prefix-free*, meaning that no codeword is the prefix of a different codeword. It’s not too hard to see that any code with this property is decodable: as you receive bits, constantly check to see if the untranslated bits correspond to a letter. As soon as they do, write down that letter and repeat.

While being prefix-free is sufficient to ensure decodability, there are decodable codes that are not prefix-free. For example, on the alphabet